

Laporan Tugas Kecil 2 IF2211 Strategi Algoritma
Voxelization Objek 3D menggunakan Octree (dengan algoritma
divide and conquer)



Fayyaz Akmal Lauda – 13524076

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

1. Pendahuluan

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

Pada Tugas Kecil 2 mata kuliah IF2211 Strategi Algoritma ini, tugas utama yang harus diselesaikan adalah mengimplementasikan sebuah program CLI (Command Line Interface) yang dapat mengonversi model 3D berekstensi .obj menjadi model .obj baru yang terdiri dari voxel. Pendekatan komputasional yang wajib digunakan dalam memecahkan masalah ini adalah algoritma Divide and Conquer.

Secara spesifik, program harus membagi ruang pembatas (Bounding Box) dari objek secara rekursif hingga mencapai tingkat kedalaman maksimum pohon (max depth) yang dinamis berdasarkan masukan pengguna. Transformasi ruang ini memiliki beberapa spesifikasi dan batasan operasional sebagai berikut:

- Surface-Only Voxelization: Transformasi hanya dilakukan pada area yang beririsan dengan permukaan objek poligon. Bagian dalam model dibiarkan kosong (hollow).
- Spatial Pruning: Algoritma harus mampu mendeteksi dan melakukan pemangkasan (pruning) pada ruang yang kosong (tidak beririsan dengan permukaan objek) untuk menghentikan penelusuran yang tidak perlu dan menghemat memori.
- Uniform Voxel Size: Output voxel harus memiliki dimensi seragam yang diekstraksi secara eksklusif dari leaf node pada batas kedalaman maksimum.

Program menerima masukan (input) berupa path direktori dari file .obj asli dan nilai kedalaman maksimum. Sebagai keluaran (output), program akan merakit file .obj hasil konversi serta menampilkan laporan statistik di terminal yang memuat metrik eksekusi algoritma, meliputi: jumlah voxel, vertex, dan faces yang terbentuk, sebaran node yang terbentuk dan di-prune pada setiap level kedalaman, kedalaman aktual, serta waktu eksekusi komputasi.

Selain spesifikasi wajib di atas, program ini juga dikembangkan dengan memanfaatkan Concurrency untuk memparalelkan proses pembelahan ruang pada level kedalaman awal untuk

mempercepat kalkulasi sebagai bentuk pemenuhan spesifikasi bonus. Eksekusi paralel ini diaplikasikan pada fase awal pembelahan Octree (pada level kedalaman atas), di mana evaluasi sub-ruang dapat dilakukan secara independen. Selanjutnya, untuk menjaga integritas struktur data dan mencegah terjadinya kondisi balapan (race condition) saat berbagai thread memperbarui metrik statistik dan menyusun daftar voxel keluaran secara bersamaan, program menerapkan mekanisme sinkronisasi memori menggunakan Mutex (Mutual Exclusion).

2. Desain Algoritma Divide and Conquer

2.1 Pemikiran Ide Desain Algoritma

Ide dasar dari penerapan algoritma Divide and Conquer pada masalah voxelization berawal dari ketidakefisienan pendekatan brute-force. Jika kita menggunakan brute-force, kita harus membagi ruang 3D menjadi sebuah grid beresolusi tinggi (misalnya $256 \times 256 \times 256$ menghasilkan jutaan titik), lalu mengevaluasi setiap kubus kecil tersebut satu per satu terhadap seluruh segitiga (faces) pada model 3D untuk melihat apakah terjadi persinggungan. Pendekatan ini memakan waktu komputasi yang sangat lambat dan memori yang masif, terutama pada model dengan puluhan ribu poligon.

Untuk mengatasi hal tersebut, algoritma Divide and Conquer digunakan melalui struktur data Octree. Alih-alih mengevaluasi jutaan kubus kecil dari awal, algoritma memulai dari satu kubus raksasa (Root Bounding Box) yang membungkus seluruh objek. Ruang ini kemudian dibelah menjadi 8 bagian yang lebih kecil secara rekursif.

Kunci utama dari ide desain ini adalah Spatial Pruning. Karena kita hanya melakukan voxelisasi pada bagian permukaan objek, sebagian besar ruang di luar objek dan di dalam perut objek adalah ruang kosong. Ketika algoritma mendeteksi bahwa sebuah sub-kubus tidak bersinggungan dengan segitiga apa pun, algoritma akan langsung memangkas (prune) cabang penelusuran tersebut. Algoritma tidak akan repot untuk membelah kubus kosong itu lagi di kedalaman selanjutnya. Dengan cara ini, pertumbuhan eksponensial $O(8^d)$ di mana d adalah kedalaman (depth), dapat ditekan secara drastis karena program hanya fokus melakukan komputasi pada area yang benar-benar memuat permukaan model 3D.

2.2 Langkah-langkah Algoritma (Devide and Conquer)

Penerapan ide algoritma Divide and Conquer dalam pembentukan Octree dilakukan melalui beberapa tahapan berikut:

1. Inisialisasi Root Bounding Box

Pertama-tama, program membaca semua titik koordinat (vertices) dari file model 3D untuk mencari nilai minimum dan maksimum di sumbu X, Y, dan Z. Dari titik-titik

ekstrem ini, program membuat satu kubus pembatas utama yang ukurannya pas melingkupi seluruh objek. Kubus ini diatur sebagai node akar (berada di kedalaman/ depth 0).

2. Fase Divide (Pembagian Ruang)

Apabila sebuah kubus (node) sedang dievaluasi dan belum mencapai batas kedalaman maksimum (*max_depth*), ruang dari kubus tersebut dibelah menjadi 8 sub-kubus (oktan) yang berukuran persis sama. Titik pusat (center) dari kedelapan anak kubus ini dikalkulasi dengan cara menggeser titik pusat kubus induk sejauh setengah dari ukuran half-dimension baru ke setiap arah kombinasi sumbu (X, Y, Z).

3. Fase Conquer (Evaluasi Persinggungan)

Setiap anak kubus yang baru saja dibentuk direpresentasikan sebagai sub-masalah baru yang harus diselesaikan. Penyelesaiannya dilakukan dengan mengevaluasi apakah volume spasial anak kubus tersebut berpotongan dengan segitiga-segitiga permukaan objek.

- Optimasi Transmisi Data: Untuk meminimalkan beban komputasi, sebuah anak kubus tidak diuji terhadap seluruh segitiga dari model 3D awal, melainkan hanya diuji terhadap himpunan segitiga yang sebelumnya telah terbukti beririsan dengan kubus induknya.

4. Fase Pruning (Pemangkasan Ruang)

Fase ini adalah mekanisme penyaringan dari fase Conquer. Jika dari hasil evaluasi interseksi ditemukan bahwa himpunan segitiga yang beririsan dengan sebuah anak kubus adalah kosong (jumlahnya 0), maka anak kubus tersebut dipastikan merupakan ruang hampa atau berada di dalam interior model yang tidak ber dinding. Penelusuran rekursif pada cabang pohon tersebut langsung dihentikan (return), dan program mencatatnya ke dalam statistik pruning.

5. Base Case (Penyelesaian dan Terminasi)

Kondisi basis (base case) dari fungsi rekursif ini tercapai apabila pembelahan kubus telah menyentuh batas *max_depth* yang diinputkan melalui parameter argumen CLI pengguna. Pada titik terdalam ini, jika kubus terbukti masih memiliki irisan dengan segitiga objek, kubus tersebut akan ditandai sebagai *IsLeaf = true* dan *IsFilled = true*. Kubus ini secara sah diakui sebagai satu unit voxel tunggal.

6. Fase Combine (Penggabungan Solusi)

Setelah seluruh pemanggilan rekursif selesai dan pohon Octree selesai dibangun, fase Combine bertugas mengumpulkan solusi-solusi kecil tersebut. Program melakukan penelusuran ulang (traversal) pada pohon dari root hingga ke leaf untuk mengumpulkan seluruh titik node yang berstatus *IsFilled*. Himpunan titik-titik voxel ini yang kemudian dikonversi kembali menjadi koordinat poligon absolut untuk ditulis ke dalam output file *.obj-voxelized* keluaran akhir.

2.3 Implemtasi Algoritma pada Kode Program

Ide desain Divide and Conquer dan mekanisme pruning diimplementasikan secara modular, dengan jantung algoritma berada pada fungsi *buildOctree* di file *octree.go*. Berikut adalah langkah algoritmanya secara langsung terhadap struktur kode:

1. Inisiasi Root Code (*main.go*)

Sebelum memasuki fungsi rekursif, program mendefinisikan ruang semesta dengan memanggil fungsi *hitungRootBox* yang mengevaluasi batas eskترم seluruh vertices model. Box ini kemudian dibungkus menjadi *rootNode* pada *depth* 0 untuk memulai penelusuran.

```
rootBox := hitungRootBox(model.Vertices)
rootNode := &OctreeNode{
    Box: rootBox, Depth: 0,
}
buildOctree(rootNode, model.Faces, model.Vertices, maxDepth, stats)
```

2. Fase Conquer & Pengecekan persinggungan (*octree.go*)

Di dalam fungsi *buildOctree*, setiap kubus (node) langsung menjalani fase Conquer. Program mengiterasi daftar segitiga dan memanggil fungsi *checkIntersection* (Separating Axis Theorem). Segitiga yang terbukti menembus volume kubus akan disaring dan dimasukkan ke dalam slice *intersectingFaces*.

```
var intersectingFaces []Segitiga
for _, face := range faces {
    v0 := vertices[face.V1]
```

```

    v1 := vertices[face.V2]
    v2 := vertices[face.V3]

    if checkIntersection(node.Box, v0, v1, v2) {
        intersectingFaces = append(intersectingFaces, face)
    }
}

```

3. Fase Pruning (octree.go)

Sejara setelah evaluasi interseksi selesai, logika pruning dijalankan. Jika *intersectingFaces* terbukti kosong (panjang slice bernilai 0), maka kubus tersebut dipastikan berada di ruang kosong. Program mencatatnya ke dalam metrik *PrunedAtDepth* secara thread-safe dan langsung menghentikan penelusuran (return).

```

    if len(intersectingFaces) == 0 {
        stats.addPruned(node.Depth)
        return
    }

```

4. Base Case (octree.go)

Jika kubus terbukti memiliki irisan poligon dan berada pada kedalaman yang setara dengan target *maxDepth*, maka proses pembelahan (Divide) dihentikan. Kubus ini diresmikan sebagai objek voxel tunggal dengan mengubah status *IsFilled* menjadi *true*.

```

    if node.Depth == maxDepth {
        node.IsLeaf = true
        node.IsFilled = true
        stats.addVoxel()
        return
    }

```

5. Fase Divide & Concurrency (octree.go)

Jika kubus beririsan namun belum mencapai max depth, ruang dibelah menjadi 8 anak (octants). Pada tahap ini, program mengkalkulasi koordinat titik pusat yang baru untuk kedelapan anak tersebut. Khusus pada depth dangkal (< 3), pemanggilan fungsi rekursif *buildOctree* untuk anak-anak tersebut didistribusikan ke dalam thread terpisah menggunakan Goroutine (*go func()*) untuk mempercepat komputasi.

```
h := node.Box.HalfDimension / 2.0
// ... (Mengkalkulasi 8 koordinat offsets) ...

for i := 0; i < 8; i++ {
    childNode := &OctreeNode{
        Box: BoundingBox{Center: offsets[i], HalfDimension: h},
        Depth: node.Depth + 1,
    }
    node.Children[i] = childNode

    if node.Depth < 3 { // Concurrency
        wg.Add(1)
        go func(child *OctreeNode) {
            defer wg.Done()
            buildOctree(child, intersectingFaces, vertices, maxDepth, stats)
        }(childNode)
    } else {
        buildOctree(childNode, intersectingFaces, vertices, maxDepth, stats)
    }
}

wg.Wait()
```

6. Fase Combine (generator.go)

Setelah seluruh pohon Octree selesai direkursi, penggabungan (Combine) diimplementasikan melalui fungsi *collectVoxels*. Fungsi ini menelusuri ulang cabang

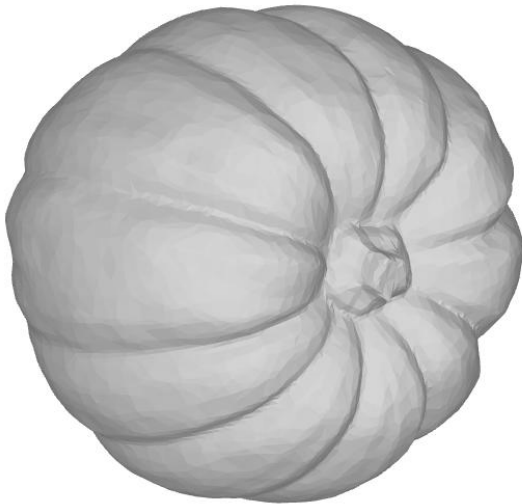
pohon dan mengekstrak semua node yang memenuhi syarat validasi akhir untuk selanjutnya ditulis ulang ke dalam file keluaran berformat *.obj*.

```
func collectVoxels(node *OctreeNode, voxels *[]BoundingBox) {  
    if node == nil {  
        return  
    }  
    if node.IsLeaf && node.IsFilled {  
        *voxels = append(*voxels, node.Box)  
        return  
    }  
    // ... (Memanggil rekursif penelusuran children)  
}
```

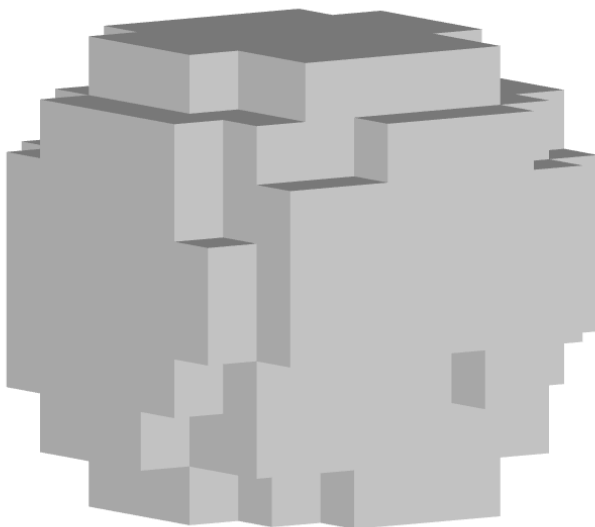
3. Hasil Uji Coba

- **Pengujian 1: Model pumpkin .obj (Max Depth: 3)**

SS Model pumpkin Asli



SS Model pumpkin-voxelized.obj depth 3



SS Terminal

```
PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\pumpkin.obj 3
Membangun Octree untuk 10000 faces...
=====
Banyaknya voxel yang terbentuk: 276
Banyaknya vertex yang terbentuk: 2208
Banyaknya faces yang terbentuk: 3312

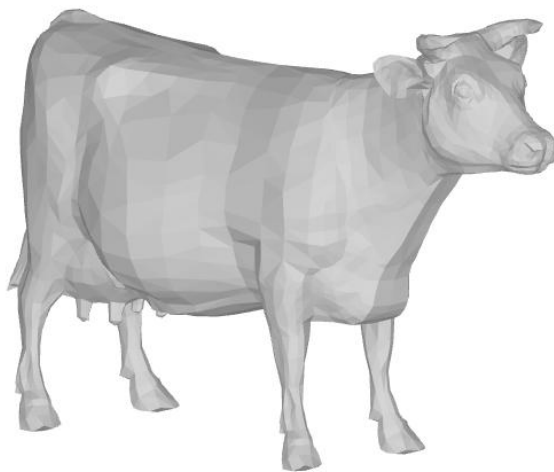
Statistik node terbentuk pada depth:
1 : 8
2 : 64
3 : 448

Statistik node yang tidak perlu ditelusuri:
2 : 8
3 : 172

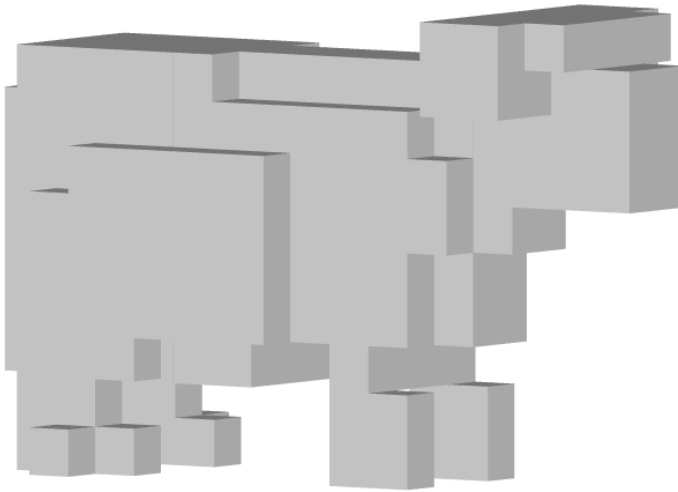
Kedalaman octree: 3
Lama waktu program berjalan: 21.9208ms
Path dimana file .obj disimpan: test\pumpkin-voxelized.obj
=====
```

- **Pengujian 2: Model cow.obj (Max Depth: 4)**

SS Model cow Asli



SS Model cow-voxelized.obj depth 4



SS Terminal

```
PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\cow.obj 4
Membangun Octree untuk 5804 faces...
=====
Banyaknya voxel yang terbentuk: 333
Banyaknya vertex yang terbentuk: 2664
Banyaknya faces yang terbentuk: 3996

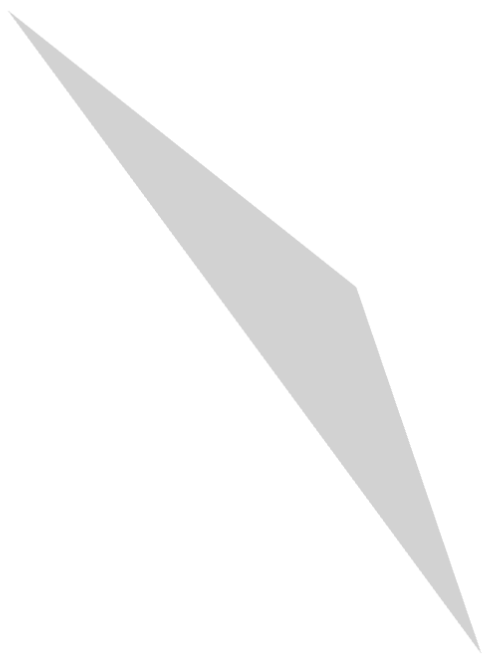
Statistik node terbentuk pada depth:
1 : 8
2 : 64
3 : 160
4 : 688

Statistik node yang tidak perlu ditelusuri:
2 : 44
3 : 74
4 : 355

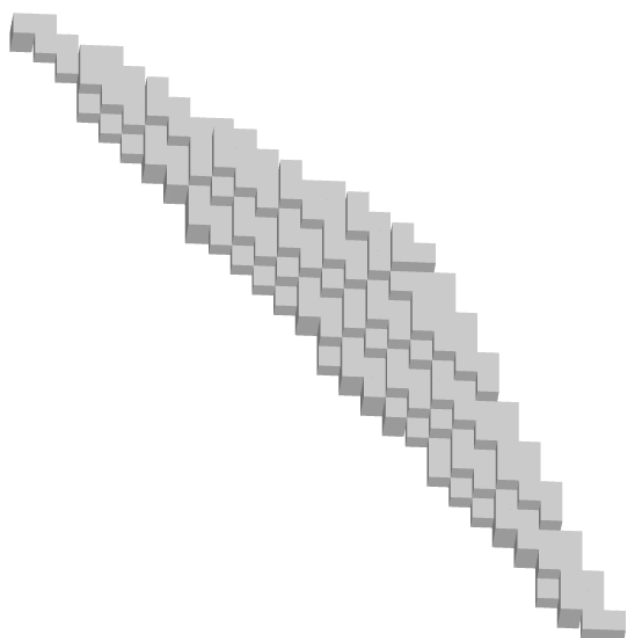
Kedalaman octree: 4
Lama waktu program berjalan: 19.016ms
Path dimana file .obj disimpan: test\cow-voxelized.obj
=====
```

- **Pengujian 3: Model line.obj (Max Depth: 5)**

SS Model line Asli



SS Model line-voxelized.obj depth 5



SS Terminal

```

PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\line.obj 5
Membangun Octree untuk 1 faces...
=====
Banyaknya voxel yang terbentuk: 280
Banyaknya vertex yang terbentuk: 2240
Banyaknya faces yang terbentuk: 3360

Statistik node terbentuk pada depth:
1 : 8
2 : 56
3 : 112
4 : 248
5 : 712

Statistik node yang tidak perlu ditelusuri:
1 : 1
2 : 42
3 : 81
4 : 159
5 : 432

Kedalaman octree: 5
Lama waktu program berjalan: 11.3799ms
Path dimana file .obj disimpan: test\line-voxelized.obj
=====

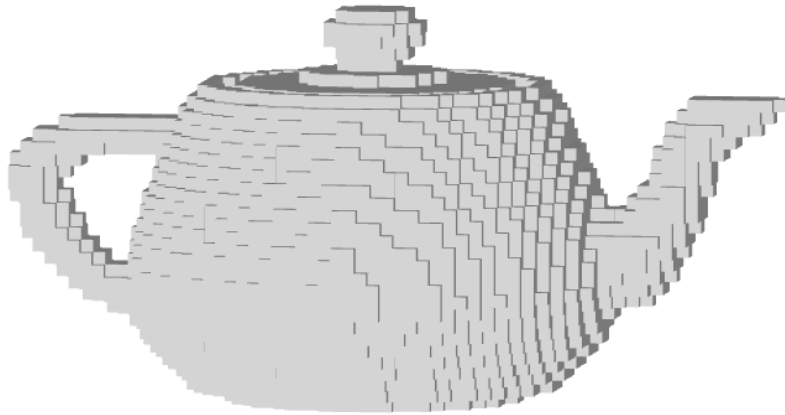
```

- **Pengujian 4: Model teapot.obj (Max Depth: 6)**

SS Model teapot Asli



SS Model teapot-voxelized.obj depth 6



SS Terminal

```
PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\teapot.obj 6
Membangun Octree untuk 1024 faces...
=====
Banyaknya voxel yang terbentuk: 7011
Banyaknya vertex yang terbentuk: 56088
Banyaknya faces yang terbentuk: 84132

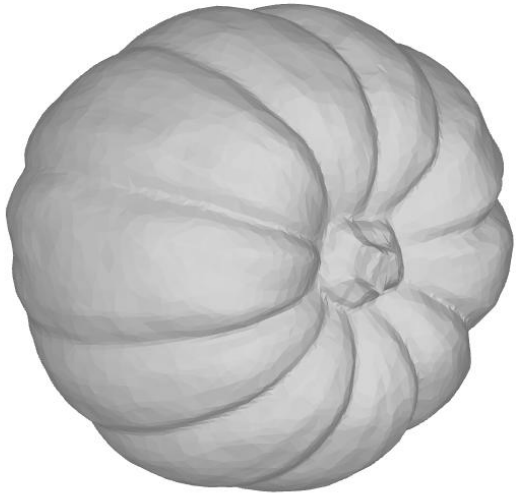
Statistik node terbentuk pada depth:
1 : 8
2 : 64
3 : 192
4 : 816
5 : 3296
6 : 13504

Statistik node yang tidak perlu ditelusuri:
2 : 40
3 : 90
4 : 404
5 : 1608
6 : 6493

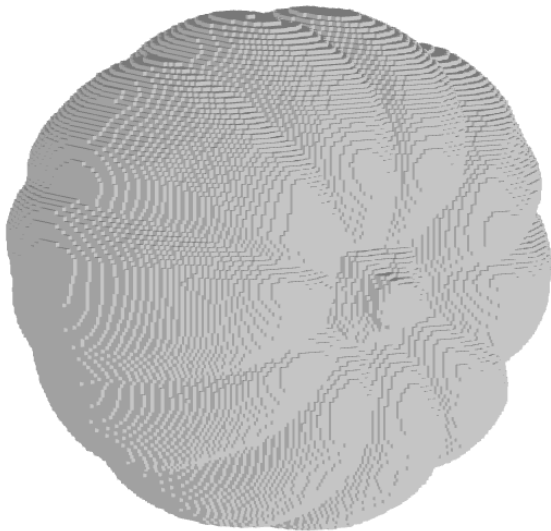
Kedalaman octree: 6
Lama waktu program berjalan: 60.698ms
Path dimana file .obj disimpan: test\teapot-voxelized.obj
=====
```

- **Pengujian 5: Model pumpkin.obj (Max Depth: 7)**

SS Model pumpkin Asli



SS Model pumpkin-voxelized.obj depth 7



SS Terminal

```

PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\pumpkin.obj 7
Membangun Octree untuk 10000 faces...
=====
Banyaknya voxel yang terbentuk: 71768
Banyaknya vertex yang terbentuk: 574144
Banyaknya faces yang terbentuk: 861216

Statistik node terbentuk pada depth:
1 : 8
2 : 64
3 : 448
4 : 2208
5 : 8648
6 : 35984
7 : 143552

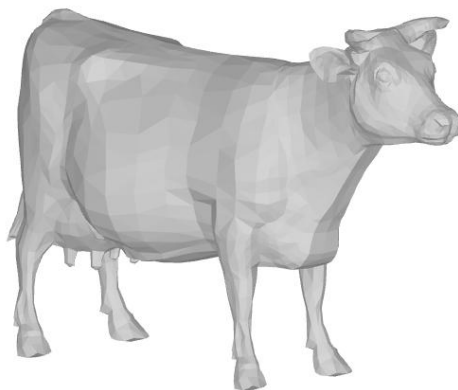
Statistik node yang tidak perlu ditelusuri:
2 : 8
3 : 172
4 : 1127
5 : 4150
6 : 18040
7 : 71784

Kedalaman octree: 7
Lama waktu program berjalan: 447.6452ms
Path dimana file .obj disimpan: test\pumpkin-voxelized.obj
=====

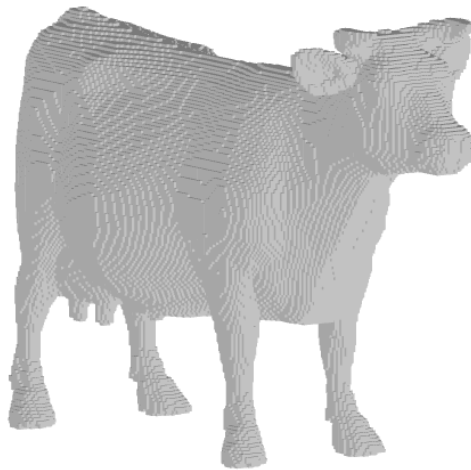
```

- **Pengujian 6: Model cow .obj (Max Depth: 8)**

SS Model cow asli



SS Model cow-voxelized.obj depth 8



SS Terminal

```
PS C:\Users\user\Documents\STIMA\Tucil2\Tucil2_13524076> .\bin\voxelizer.exe test\cow.obj 8
Membangun Octree untuk 5804 faces...
=====
Banyaknya voxel yang terbentuk: 88306
Banyaknya vertex yang terbentuk: 706448
Banyaknya faces yang terbentuk: 1059672

Statistik node terbentuk pada depth:
1 : 8
2 : 64
3 : 160
4 : 688
5 : 2664
6 : 10680
7 : 43272
8 : 176272

Statistik node yang tidak perlu ditelusuri:
2 : 44
3 : 74
4 : 355
5 : 1329
6 : 5271
7 : 21238
8 : 87966

Kedalaman octree: 8
Lama waktu program berjalan: 506.9331ms
Path dimana file .obj disimpan: test\cow-voxelized.obj
=====
```

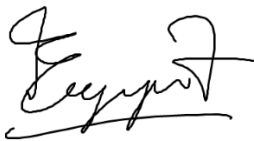
4. Tautan dan Lampiran

Link repository Github: https://github.com/pepayaz/Tucil2_13524076

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan concurrency	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

5. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Fayyaz Akmal Lauda